

Contents

1	Categories management	1
1.1	PHP Categories Class	1
1.2	Explanation	1
1.3	HTML Forms	2
1.3.1	Form to Add a New Category (add_category.html)	2
1.3.2	PHP Script to Handle the Form (add_category.php)	2
1.3.3	Displaying Categories (list_categories.php)	2
1.4	Security Considerations	3

1 Categories management

Let's create a `Categories` class in PHP, which will manage the CRUD operations for the categories in the To-Do application. Along with the PHP class, I'll provide an example of HTML forms for adding and viewing categories.

1.1 PHP Categories Class

```
class Categories {
    private $pdo;

    public function __construct($pdo) {
        $this->pdo = $pdo;
    }

    public function create($name) {
        $sql = "INSERT INTO categories (name) VALUES (?)";
        $stmt = $this->pdo->prepare($sql);
        $stmt->execute([$name]);
    }

    public function getAll() {
        $sql = "SELECT * FROM categories";
        $stmt = $this->pdo->query($sql);
        return $stmt->fetchAll();
    }

    // Additional methods for updating and deleting categories can be added here
}
```

1.2 Explanation

- **Constructor:** Initializes the database connection.
- **Create Category:** Adds a new category to the database.

- **Get All Categories:** Retrieves all categories from the database.

1.3 HTML Forms

1.3.1 Form to Add a New Category (add_category.html)

```
<!DOCTYPE html>
<html>
<head>
    <title>Add New Category</title>
</head>
<body>
    <h1>Add New Category</h1>
    <form action="add_category.php" method="post">
        Category Name: <input type="text" name="name" required><br>
        <input type="submit" value="Add Category">
    </form>
</body>
</html>
```

1.3.2 PHP Script to Handle the Form (add_category.php)

```
<?php
require_once 'Categories.php';
require_once 'db_connect.php'; // PDO connection

$categories = new Categories($pdo);

if ($_SERVER["REQUEST_METHOD"] == "POST") {
    $categories->create($_POST['name']);
    echo "Category added successfully.";
    // Redirect or show a success message
}
```

1.3.3 Displaying Categories (list_categories.php)

```
<?php
require_once 'Categories.php';
require_once 'db_connect.php';

$categories = new Categories($pdo);
$allCategories = $categories->getAll();
?>

<!DOCTYPE html>
<html>
<head>
```

```
<title>Category List</title>
</head>
<body>
  <h1>Category List</h1>
  <ul>
    <?php foreach ($allCategories as $category): ?>
      <li><?php echo htmlspecialchars($category['name']); ?></li>
    <?php endforeach; ?>
  </ul>
</body>
</html>
```

1.4 Security Considerations

- **Input Validation:** Always validate and sanitize inputs to prevent SQL injection and other forms of attacks.
- **Error Handling:** Implement proper error handling, especially for database operations.
- **User Permissions:** Depending on your application, you might want to restrict category management to certain user roles (like admin).

This class and the corresponding HTML forms provide basic functionality for managing categories in our To-Do application. In a full application, you'd also include methods for updating and deleting categories, along with the necessary HTML forms and PHP scripts.